Structured-Grid SOR with Temporal Blocking:
from one Xeon thread to a CUDA-tiled L40S

Che-Yu Yu
EC527, Spring 2026

April 30, 2026

1 Description of the Algorithm

1.1 Mathematical foundation

We solve the 2D Laplace equation on the unit square with Dirichlet boundary data:

$$\nabla^2 u(x,y) = 0, \qquad u\big|_{\partial\Omega} = g(x,y). \tag{1}$$

On a uniform $N\times N$ grid with spacing $h{=}1/(N{-}1)$, the second-order centred difference rearranges to the canonical 5-point stencil shown in Figure 1:

$$u_{i,j} = \tfrac{1}{4}(u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1})\,. \tag{2}$$

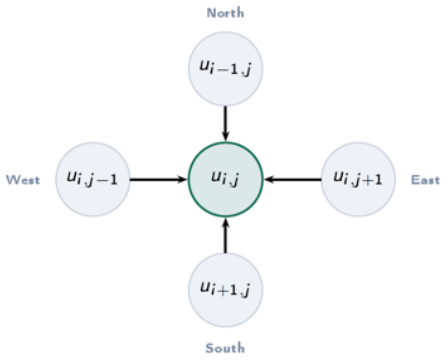


Figure 1: The 5-point Laplacian stencil. Each interior cell averages its four orthogonal neighbours.

Equation (2) is the Jacobi update. Successive Over-Relaxation (SOR) adds a relaxation parameter ω to the Gauss-Seidel update:

$$u_{i,j}^{(k+1)} = (1-\omega)\,u_{i,j}^{(k)} + \frac{\omega}{4}\Big(u_{i-1,j}^{(k+1)} + u_{i+1,j}^{(k)} + u_{i,j-1}^{(k+1)} + u_{i,j+1}^{(k)}\Big)\,. \tag{3}$$

The right-hand side mixes already-updated and not-yet-updated neighbours. For the Poisson problem on a uniform grid, the optimal relaxation parameter is

$$\omega_{\text{opt}} = \frac{2}{1+\sin\!\big(\pi/(N{-}1)\big)} \;\approx\; 2 - \frac{2\pi}{N}. \tag{4}$$

That gives $O(N)$ iterations for tuned SOR vs $O(N^2)$ for Jacobi. We verify this experimentally in 2.5.2.

1.2 The specific algorithmic form used

We implemented three forms of the sweep. The *baseline* is a straight scan-order rewrite of (3):

```
/* baseline SOR, scan order, in-place: sequential */
for (i = 1; i < N-1; i++)
  for (j = 1; j < N-1; j++) {
    sum = u[i-1][j] + u[i+1][j] + u[i][j-1] + u[i][j+1];
    u[i][j] = (1.0f - OMEGA) * u[i][j] + (OMEGA * 0.25f) * sum;
  }
```

The loop-carried dependency forbids parallelising the outer loop directly. Two ways out:

Red-black SOR. Colour the grid like a checkerboard. Red cells depend only on black cells, so each colour can be updated in parallel:

```
for (parity = 0; parity < 2; parity++) {
  #pragma omp parallel for
  for (i = 1; i < N-1; i++) {
    int j0 = ((i + parity) & 1) ? 1 : 2;
    for (j = j0; j < N-1; j += 2) {
      sum = u[i-1][j] + u[i+1][j] + u[i][j-1] + u[i][j+1];
      u[i][j] = (1.0f - OMEGA) * u[i][j] + (OMEGA * 0.25f) * sum;
    }
  }
}
```

Used in `sor2d_rb.c` for the omega-tuning study (2.5.2).

Jacobi ping-pong. Read from `src`, write to `dst`, swap. No within-sweep dependency:

```
#pragma omp parallel for
for (i = 1; i < N-1; i++)
  for (j = 1; j < N-1; j++) {
    nb = 0.25f * (src[i-1][j] + src[i+1][j] + src[i][j-1] + src[i][j+1]);
    dst[i][j] = src[i][j] - OMEGA * (src[i][j] - nb);
  }
swap(&src, &dst);
```

We trade convergence rate (stable only for $\omega \in (0, 1]$ on large grids; we use ω=0.9) for clean parallelism and the ability to do temporal blocking.

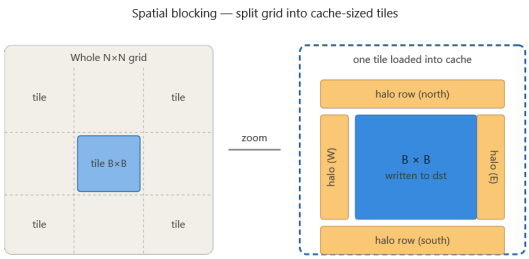
Temporal blocking (the shrinking trapezoid). For each super-step of T Jacobi sweeps:

1. Tile the grid into $B \times B$ output regions (Figure 2).
2. For each tile, load a $(B+2T)^2$ scratch buffer.
3. Run T in-scratch Jacobi sweeps; the valid region shrinks by one cell per side per sub-step (Figure 2).
4. Write the central B^2 back to `dst`.

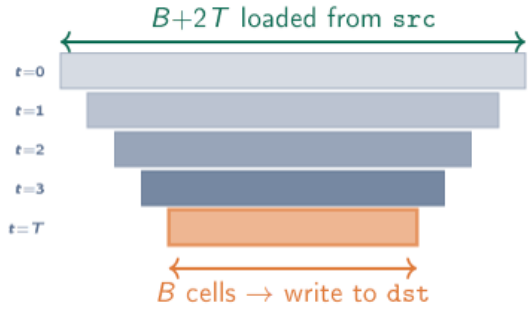
DRAM traffic for the central cells drops by a factor of T. The cost is redundant halo work: the work multiplier is $((B+2T)/B)^2$. We measure its unimodal optimum in 2.5.

Variable glossary:

- N. Grid side; interior is $(N-2)^2$ cells.
- B. Spatial block size.
- T. Temporal halo (sub-sweeps fused per DRAM round-trip).



(a) Spatial blocking: per-thread $B \times B$ tiles with a 1-cell halo.



(b) Temporal blocking: load $B+2T$ cells, do T sub-steps; the valid region shrinks by one cell per side per sub-step.

Figure 2: Spatial vs temporal blocking.

- ω. Relaxation factor. 0.9 for ping-pong kernels; $\omega_{\text{opt}} \approx 2 - 2\pi/N$ for red-black.
- iters. Total sweep count; must satisfy `iters % T == 0` for temporal kernels.
- TOL. Convergence tolerance (10^{-5}).

1.3 Implementation reality check

Two paths to parallelism. Scan-order SOR has a loop-carried dependency, so we use either red-black SOR (parallel within each colour, ω_{opt} via (4)) or damped Jacobi ping-pong (pure parallelism but $\omega \leq 1$ on large grids). Red-black is used in 2.5.2 for the algorithmic-tuning story; Jacobi ping-pong is used everywhere else because it composes cleanly with temporal blocking. The honest apples-to-apples comparison at the end is per-iter throughput (CPE), not iters-to-tolerance.

Validation strategy. Three layers:

1. Every CPU temporal kernel is bit-identical to its in-binary baseline (smoke runs print `max|base-temp| = 0.0000e+00`).
2. The GPU temporal kernel agrees with the GPU baseline at $\sim 1.4 \times 10^{-7}$ relative (FP32 round-off, well under TOL).
3. The omega-sweep in 2.5.2 reproduces ω_{opt} to 0.1% at N=128, sanity-checking the discretisation, boundaries, and convergence detector.

2 Scalar CPU Version

2.1 The hot inner loop

The 5-point stencil is the only kernel worth optimising; it executes $N^2 \cdot \text{iters}$ times. Per cell we have $\sim$6 FLOPs against 8 B/cell of DRAM traffic at FP32, giving an arithmetic intensity of $\approx$ 0.75 FLOP/byte. That puts us in the bandwidth-bound regime (Figure 3), so every optimisation in this project is a memory optimisation in disguise.

Inner-loop micro-variants. At N=512, 1 thread, 64 iters: `ij`-order gives CPE 0.39 with or without `restrict`; `ji`-order jumps to 1.84 from cache thrashing on stride-N access. We keep `ij`-order.

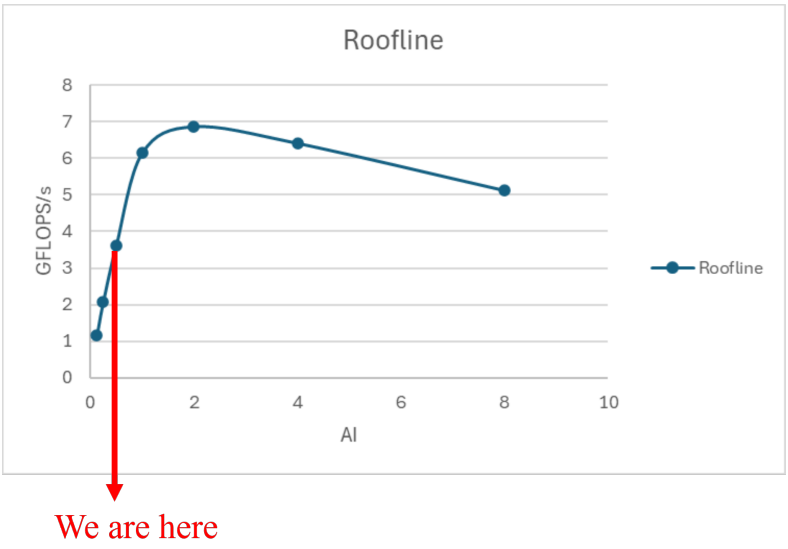


Figure 3: Roofline-style throughput curve. AI ≈ 0.5–0.75 puts the stencil where the red arrow lands. On the bandwidth-bound diagonal, well below the peak-compute plateau.

2.2 The full sweep

Three forms in increasing order of complexity:

1. **Plain double-loop** (the `baseline` in every binary).
2. **Spatially blocked.** Tile into $B \times B$ chunks (Figure 2). Helps once the working set exceeds L2.
3. **Temporally blocked.** Shrinking trapezoid (Figure 2); per-thread scratch is $2(B+2T)^2$ floats.

At B=128, T=8 the work multiplier is $(144/128)^2 \approx 1.27\times$. The bandwidth savings have to outpace this overhead for temporal blocking to win.

2.3 Loose ends

- **Boundary copies.** Pass-through every sweep. Negligible vs interior data.
- **Per-tile scratch malloc.** The OMP version does `malloc(2*(B+2T)*(B+2T)*sizeof(float))` once per thread per super-step. At small N this is real overhead (Figure 4, where temporal loses to baseline at $N \leq 512$).
- **Convergence-norm reduction.** Used only in `sor2d_rb.c` (one extra reduction per cell); responsible for slower per-iter timings in 2.5.2.
- **Final swap.** A pointer swap, not memcpy.

2.4 Stitching it together (real code)

The actual temporal-blocking inner loop from `sor2d_omp.c` (stripped of declarations):

```
#pragma omp for collapse(2) schedule(static)
for (ti = 0; ti < ntiles_i; ti++) {
  for (tj = 0; tj < ntiles_j; tj++) {
    int i0 = 1 + ti*B, j0 = 1 + tj*B; /* tile origin in dst */
    int gi0 = i0 - T, gj0 = j0 - T; /* scratch origin */
```

```
    /* (1) load (B+2T)x(B+2T) scratch from src */
    for (si = 0; si < S; si++)
      for (sj = 0; sj < S; sj++) {
        sa[si*S+sj] = src[clamp(gi0+si)*N + clamp(gj0+sj)];
      }
    memcpy(sb, sa, sizeof(float)*S*S);

    /* (2) T sub-steps on the in-scratch trapezoid */
    A = sa; Bp = sb;
    for (t = 0; t < T; t++) {
      memcpy(Bp, A, sizeof(float)*S*S);
      for (si = 1+t; si < S-1-t; si++)
        for (sj = 1+t; sj < S-1-t; sj++) {
          float s = A[si*S+sj];
          float nb = 0.25f*(A[(si-1)*S+sj] + A[(si+1)*S+sj]
                          + A[si*S+sj-1] + A[si*S+sj+1]);
          Bp[si*S+sj] = s - OMEGA*(s - nb);
        }
      tmp = A; A = Bp; Bp = tmp;
    }

    /* (3) write central B x B back to dst */
    for (si = T; si < T+B; si++)
      memcpy(dst + (gi0+si)*N + (gj0+T),
             A + si*S + T,
             B*sizeof(float));
  }
}
```

`collapse(2)` flattens the tile work-list so OMP can load-balance. The inner `memcpy(Bp, A, ...)` preserves the stale-halo perimeter so the compute loop is branch-free and the compiler vectorises it cleanly.

2.5 Code and Results

2.5.1 Discussion of CPE

Size sweep. Figure 4 shows the cache-hierarchy story.
At B=128, T=8 the per-thread scratch is 81 KB and fits in L1 even at N=4098. The 2× ratio matches the bandwidth ratio between cache and DRAM, give or take halo overhead.

Block size sweep. At fixed N=2048 we sweep B (Figure 5).
We pick B=128: middle of the plateau, scratch fits L2, and $2046/128 = 16$ tiles per axis aligns cleanly.

Halo / T sweep. At fixed B=128 we sweep T (Figure 6).

Strong scaling and decomposition. Figure 7 shows strong scaling: temporal hits 91% of perfect at 8 threads (7.31×); baseline only 56% (4.49×). At 1 thread temporal barely beats baseline (1.08×). Temporal blocking earns its keep only once parallelism stresses DRAM.

Strip-persistent wins; interleaved loses to false sharing; 2D-block sits in the middle. The `spawn`-vs-`persistent` gap is largest for strip (1.54× slowdown) because strip's per-sweep work is smallest.

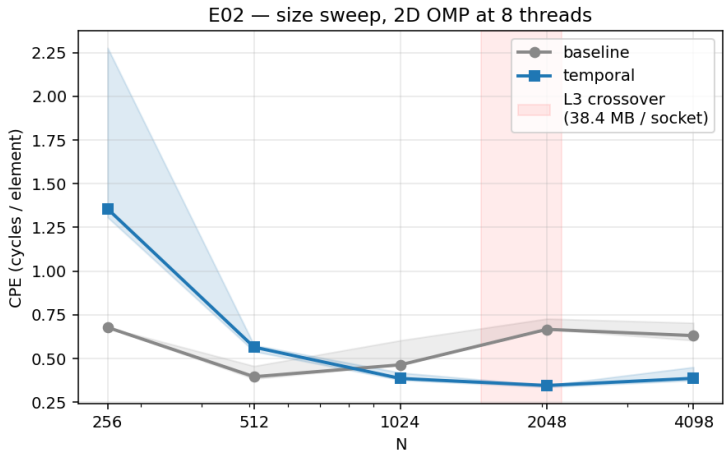


Figure 4: 2D OMP size sweep at 8 threads, B=128, T=8, iters=64. The L3→DRAM crossover lands between N=1024 and N=2048: baseline CPE rises from 0.464 to 0.667 as the working set exceeds 38 MB. Temporal stays at ∼0.35 across N=1024 – 4098 (L2-resident on its scratch) and wins 1.93× at N=2048. Below L3 temporal loses: per-tile malloc + halo work cancel the savings on a working set the baseline already had in cache.

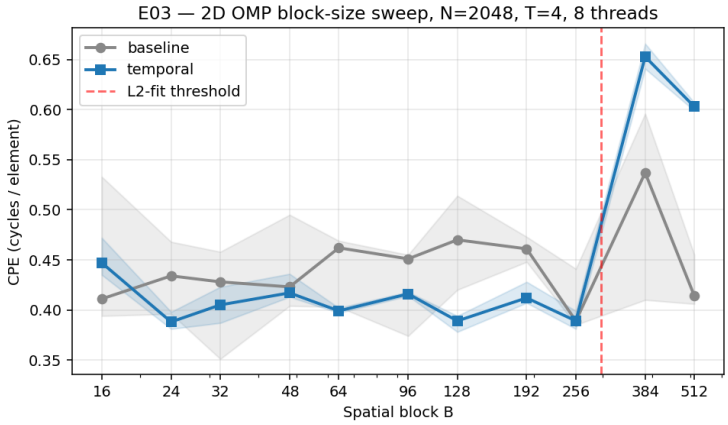


Figure 5: Block-size sweep at N=2048, T=4, 8 threads. Temporal has a wide plateau across $B \in [24, 256]$ at CPE ∼0.39, then falls off a cliff at $B \geq 384$. Red dashed line is the L2-fit threshold ($B \leq 297$); beyond that, scratch spills L2.

Pthread spawn-overhead. Figure 8 is the direct Lab-6-Part-1b answer. The per-sweep `pthread_create` cost is roughly 153 μs at 4 threads, or about 38 μs per `pthread_create+join` pair. Spawning per sweep is a 1.89× tax on the kernel. This is why OpenMP under the hood uses a persistent thread pool.

The OMP-vs-pthreads surprise. OMP-8t baseline measured CPE 0.667 at N=2050, while pthreads-strip-persistent at the same N measured ∼0.4. Pthreads beat OpenMP on the baseline sweep on this machine. Likely a NUMA / first-touch interaction across the two sockets; we didn't chase it further.

2.5.2 Discussion of Iteration Number and Error

The previous section was per-iter throughput. This one is iters-to- tolerance, the algorithmic story.

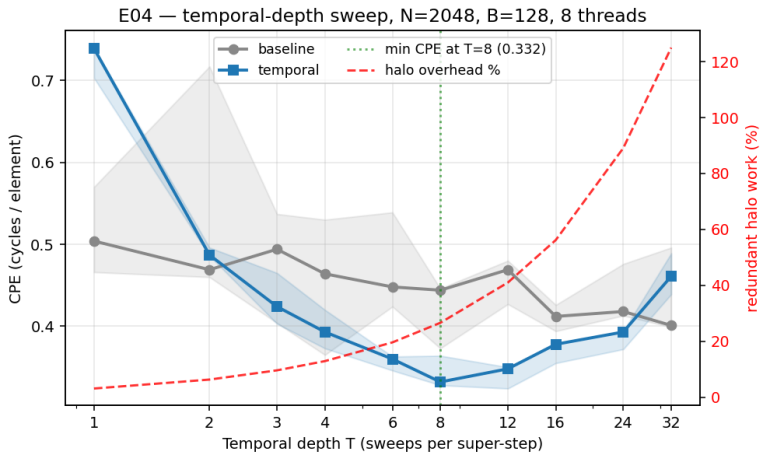
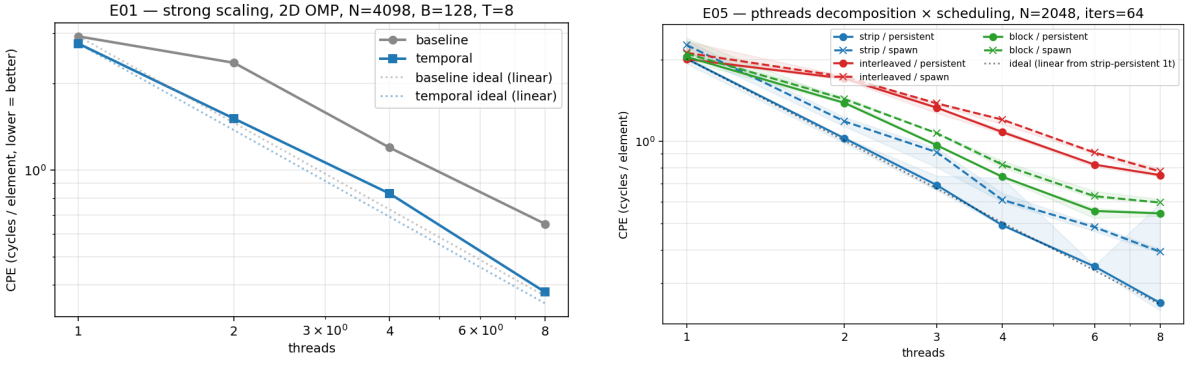


Figure 6: Temporal depth T sweep at N=2048, B=128, 8 threads, iters=96. Left axis: CPE. Right axis: redundant halo work, $((B+2T)^2/B^2 - 1)\cdot 100$. Optimum at T=8, CPE 0.332.

T	baseline CPE	temporal CPE	temp/base	halo overhead
1	0.504	0.739	1.47×	3%
2	0.469	0.487	1.04×	6%
4	0.464	0.393	0.85×	13%
6	0.448	0.360	0.80×	20%
8	0.444	**0.332**	**0.75**×	27%
12	0.469	0.348	0.74×	41%
16	0.412	0.378	0.92×	56%
24	0.418	0.393	0.94×	89%
32	0.401	0.461	1.15×	125%

Table 1: Halo/T sweep. Best at T=8 (25% faster than baseline); T=1 pays scratch overhead with no DRAM-reuse benefit; T=32 is swallowed by halo overhead.



(a) Strong scaling.

(b) Pthreads decomposition × schedule.

Figure 7: Left: strong scaling at N=4098, B=128, T=8. Right: 3 modes (strip / interleaved / 2D-block) × 2 schedules (persistent / spawn). Strip-persistent at 8 threads: CPE 0.255, 7.89× scaling. Interleaved bottoms out at 2.67× from false sharing on adjacent rows.

The cliff at $\omega > \omega_{\text{opt}}$ is sharp: ω=1.99 already gives back most of the win (1,230 iters). Above ω=2 the iteration is unconditionally divergent.

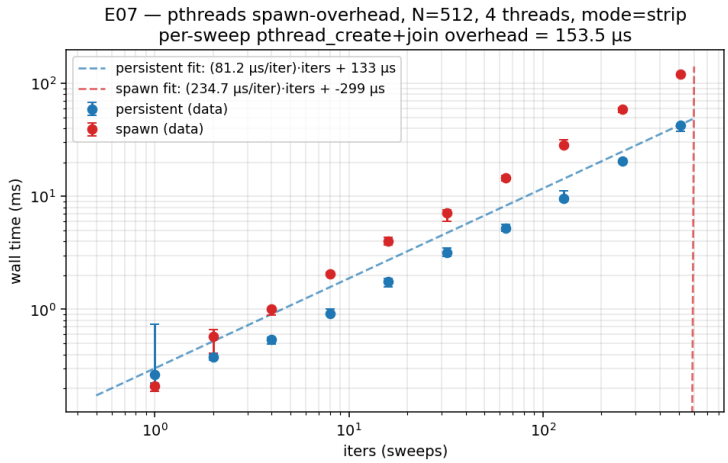


Figure 8: Wall-time vs iters at N=512, 4 threads, mode=strip. Persistent slope: 81 μs/sweep (compute work). Spawn slope: 235 μs/sweep. Slope difference 153 μs is the per-sweep `pthread_create+join` overhead, larger than the compute work itself.

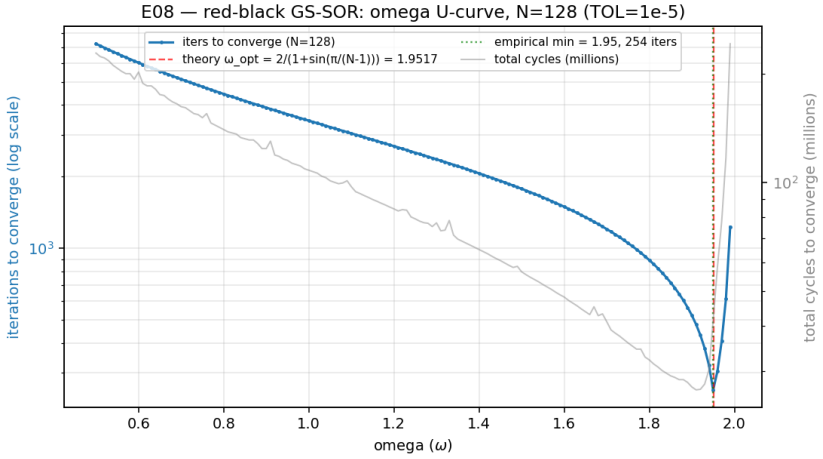


Figure 9: Red-black GS-SOR omega U-curve at N=128, TOL=10^{-5}. Theory $\omega_{\text{opt}} = 1.9517$ (red dashed) matches empirical minimum at $\omega = 1.95$ (green dotted) to 0.1%. 254 iters at ω_{opt} vs 3,434 at ω=1 (Gauss-Seidel) is a 13.5× fewer-iteration win.

This is the project's biggest single algorithmic optimisation, and the temporal-blocking kernels can't directly exploit it: damped Jacobi at $\omega = 0.9$ lives on the under-relaxation side of the curve; ω>1 in the ping-pong form has spectral radius > 1 on large grids and diverges. The natural follow-up is a temporal-blocking variant of red-black GS-SOR, which is the headline future-work item.

FP32 round-off. At N=128 and 254 iters the forward error stays around 10^{-6}, comfortably below TOL. At larger N or tighter TOL we'd need Kahan compensation or FP64.

2.6 Overall comments

1. **Temporal blocking only earns its keep in the bandwidth-bound regime.** Single-thread CPU was a wash (1.08×); 8-thread + DRAM-spilling working set was 2×.
2. **The T-sweet-spot is sharp.** T=8 wins by 25%; T=1 loses to scratch overhead; T=32

loses to halo work.

3. **Strip wins for 2D pthreads**, with 7.89× scaling at 8 threads.
4. **OMP can be slower than pthreads at the baseline sweep on a NUMA box.** Suspect: first-touch + cross-socket threads.
5. **ω_{opt} is real and matters more than any per-iter optimisation we tried on CPU.** 13.5× fewer iters at N=128.

3 To the GPU!

The L40S has ∼30× more memory bandwidth than one Xeon socket (864 vs ∼280 GB/s) and far more parallelism. The question is whether temporal blocking's relative advantage shrinks on the GPU. Spoiler: yes, it does.

3.1 Memory allocation and transfer

buffer	**size**	**at N=2048**	**H→D once?**
`d_src`	$N^2 \cdot 4$ B	16 MB	yes (init)
`d_dst`	$N^2 \cdot 4$ B	16 MB	no (alloc only)
shared-mem scratch (per block)	$2 \cdot \text{TILE}^2 \cdot 4$ B	8 KB	no (per kernel)
constants	negligible		yes

Table 2: GPU memory layout at N=2048, FP32. The dominant transfer cost is the initial 16 MB H→D copy of `d_src` (∼0.6 ms at PCIe Gen4); after that, the iteration loop is fully on-device.

We don't run a residual reduction inside the temporal kernel, so the binary uses fixed iters rather than TOL-based stop. A real solver would need a separate per-super-step reduction kernel.

3.2 The parallelised hot loop on the GPU

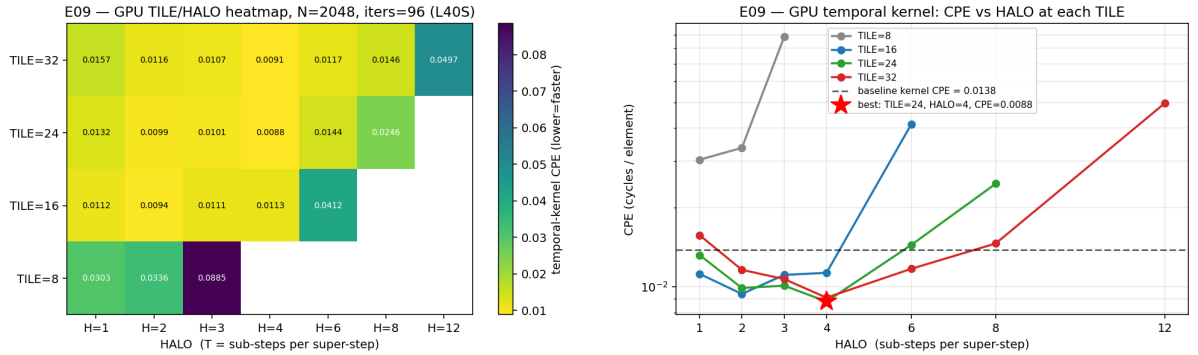
Vocabulary. CPU "thread" maps to a CUDA *block* (private shared memory, `__syncthreads()`); the CPU $B \times B$ tile maps to a CUDA TILE; warps inside a block are the SIMD-lane equivalent.

```
__global__ void sor_temporal(const float *d_old, float *d_new,
                             int N, float omega, int tile, int halo)
{
  extern __shared__ float sshared[];
  float *sa = sshared, *sb = sshared + tile*tile;

  /* (1) clamp-to-edge load into both ping-pong buffers */
  load_tile_with_halo(sa, sb, d_old);
  __syncthreads();

  /* (2) HALO sub-steps; valid region shrinks 1 per side per step */
  for (int t = 0; t < halo; t++) {
    if (in_trapezoid(t)) compute_5pt_into_other_buffer();
    else carry_through();
    __syncthreads();
  }
```

```
  /* (3) write central INTER x INTER cells back to d_new */
  if (in_central_region) d_new[...] = result_from_current_buffer;
}
```

The kernel uses dynamic shared memory so we can vary TILE/HALO at runtime. At TILE= 24 each block has 576 threads (18 warps), and the SM can fit two blocks at once. Good for occupancy.

Tuning sweep. Figure 10 is the (TILE, HALO) heatmap.



(a) Heatmap; dark = best CPE.

(b) CPE vs HALO, one line per TILE.

Figure 10: GPU 2D temporal sweep at N=2048, iters=96. Best at (TILE=24, HALO=4) with CPE 0.0088, **1.56**× over the per-launch baseline (CPE 0.0137). Empirical rule: **HALO** $\leq$ **INTER/4**.

3.3 Partitioning the rest of the algorithm

We had a real design choice between two GPU kernel structures:

1. **One kernel per stencil sub-step** (`sor_sweep`). Easy to add a residual reduction between launches; pays $N^2 \cdot$ iters DRAM round-trips and one $\sim$30 μs launch each time.
2. **One temporal-blocked kernel** (`sor_temporal`). Holds a (TILE)^2 shared-memory tile and does HALO sub-steps per launch. Fewer DRAM trips and fewer launches, but mid-super-step residual checks are awkward.

Both run side-by-side in the same binary. Shared memory at TILE=24/HALO=4 is $2{\cdot}24^2{\cdot}4 =$ 4.5 KB per block, well under the 100 KB cap.

3.4 Running it end-to-end

3.5 Discussion of GPU results

The temporal/baseline ratio is smaller on GPU. On CPU we got 1.93× at the sweet spot; on GPU only 1.56×. The L40S has a $\sim$96 MB L2, which already fits our $2N^2$ buffer at N=2050, so the baseline kernel is mostly cache-resident and temporal blocking has little to recover. The genuine memory-saving win re-opens at N=4098 (128 MB working set exceeds L2), where the ratio is 2.36×.

Apples to apples. Quoted speedup vs the serial-CPU baseline is 310×. That's the headline but it isn't the honest within-tier number. Honest comparisons:

- GPU-temporal vs OMP-temporal: 0.332/0.009 $\approx$ **37**×. "GPU is better than 8-core CPU."

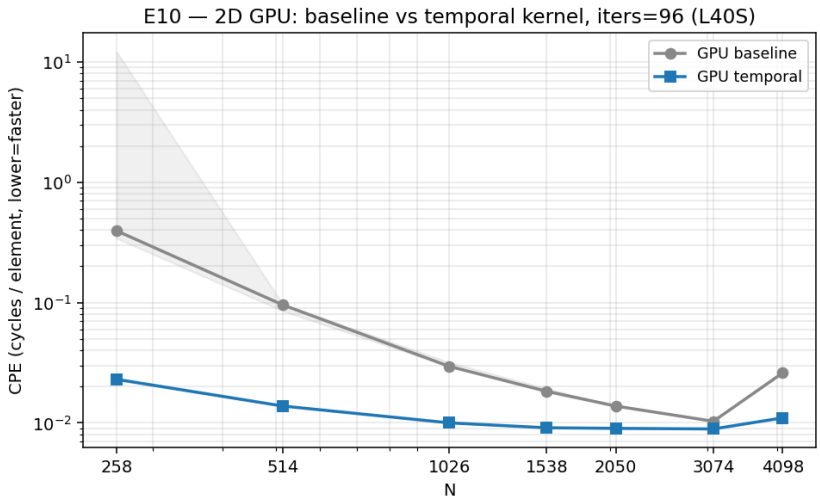


Figure 11: 2D GPU baseline vs temporal across N, iters=96, defaults TILE=32, HALO=4. At small N the baseline is launch-overhead- dominated; at $N \geq 1024$ the steady-state speedup is a more honest **1.5–3**×. At N=4098 both kernels degrade (working set 128 MB exceeds L40S's ∼96 MB L2). Best 2D GPU CPE = **0.0089** at N=3074.

N	baseline ms	temporal ms	baseline CPE	temporal CPE
258	2.7	0.16	0.395	0.023
514	1.8	0.27	0.096	0.014
1026	2.4	0.83	0.030	0.010
1538	3.7	1.86	0.018	0.0091
2050	5.7	3.83	0.014	0.0090
3074	9.5	8.18	0.010	**0.0089**
4098	43.6	18.4	0.026	0.011

Table 3: 2D GPU N-sweep, iters=96. CPE values reported at CPNS=2.0 (lab convention).

- GPU-temporal vs GPU-baseline: **1.56**×. "Shared-memory temporal blocking helps."
- GPU-baseline vs serial-CPU baseline: **186**×. "L40S is much better than one Xeon thread."

The 310× is the product $186 \cdot 1.56$, presented on its own it overstates the within-architecture optimisation.

The 3D GPU honest negative. The 3D temporal kernel (`sor3d_gpu.cu`, TILE=8, HALO=2) loses to the 3D baseline at $N \geq 194$. The 3D halo overhead is 7× redundant work at this configuration, much worse than the 2D case. The standard fix (2.5D streaming) is on the future-work list.

Cross-tier headline. At N=2050, iters=96, 8 threads:
At CPNS=2.0, GPU temporal's 0.009 CPE corresponds to about 4.5 ps per element. There's no easy 2× left without changing the algorithm.

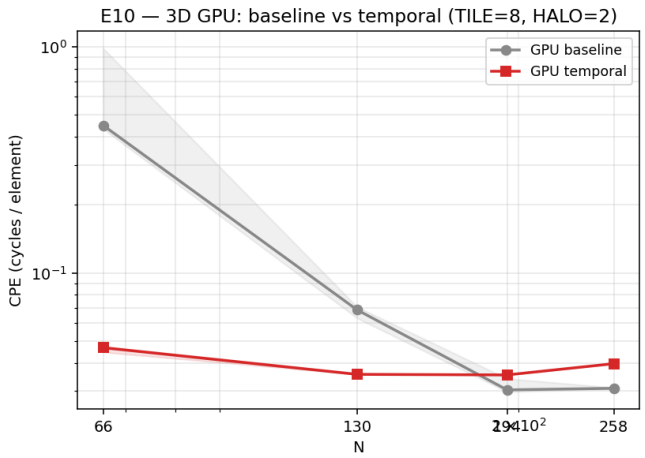


Figure 12: 3D GPU baseline (grey) vs temporal (red) across N. At $N \geq 194$ the 3D halo overhead (7× redundant work) eats the DRAM savings.

tier / kind	**CPE**	**vs serial baseline**
serial-CPU baseline	2.792	1.00×
serial-CPU temporal	2.586	1.08×
pthreads-8t-strip persistent	0.399	7.0×
OMP-8t baseline	0.667	4.2×
OMP-8t temporal	0.332	8.7×
GPU baseline	0.015	186×
GPU temporal	**0.009**	**310×**

Table 4: Cross-tier CPE summary.

4 Closing Thoughts

What worked. Temporal blocking on the bandwidth-bound CPU regime was textbook: 1.93× at N=2048. Every CPU config was bit-identical to its baseline. GPU shared-memory tiling was clean: 1.56× over the per-launch baseline at the right (TILE, HALO) sweet spot.

What didn't. Three things in increasing severity:

1. **Single-thread CPU temporal was a wash** (1.08×). Per-tile malloc + halo work cancelled the bandwidth savings on a core where DRAM was already lightly loaded. Not a flaw, but worth being upfront about.
2. **OMP baseline (CPE 0.667) was worse than pthreads (CPE 0.399)** at N=2050, 8 threads. Best guess: first-touch NUMA + cross-socket spreading. Honest unanswered.
3. **3D GPU temporal lost to baseline at $N \geq 194$.** Cubic tile ⇒ surface-to-volume gets worse cubically; with TILE=8/HALO=2 the work multiplier is 7×. The fix (2.5D streaming) didn't make the cut.

Precision. (1) Lab-7's ω=1.85 in damped-Jacobi-ping-pong diverges on large grids; fixed by switching to ω=0.9 and flagging non-finite values explicitly. (2) FP32 round-off stays under TOL at every config we ran. (3) The N-dependence of ω_{opt} in 2.5.2 matches (4).

Future directions.

1. **Red-black GS-SOR temporal kernel.** Combine the 13.5× algorithmic win with the 1.56× per-iter win. Re-derive the trapezoid for paired half-sweeps. Expected ∼30× over the current GPU temporal.
2. **2.5D-streaming for 3D GPU temporal** (Micikevicius 2009). Slide a 3-z-plane window through shared memory; turns 3D's 7× halo into 2D's 0.78×.
3. **Multigrid V-cycle.** The proper way to amplify SOR's per-iter speedup with a 5–20× fewer-iterations win on top.
4. *(Easter egg) Mixed precision.* FP64 residual accumulator with FP16/BF16 inner stencil; iterative-refinement style. Framework left commented out in the source.

5 Compilation instructions

```
# CPU binaries
gcc -O1 -std=gnu11 -fopenmp src/sor2d_omp.c -o build/sor2d_omp -lrt -lm
gcc -O1 -std=gnu11 -pthread src/sor2d_pth_decomp.c -o build/sor2d_pth_decomp \
                                          -lpthread -lrt -lm

# GPU binaries
module load cuda/12.8
nvcc -O2 src/sor2d_gpu.cu -o build/sor2d_gpu
nvcc -O2 src/sor3d_gpu.cu -o build/sor3d_gpu

# Or just 'make && make gpu'.
```

`-O1` matches the EC527 lab convention (keeps gcc from auto-vectorising and complicating CPE analysis). `-O3 -march=native` would roughly halve baseline CPE on this CPU; we chose course-style consistency over peak performance.

Preprocessor knobs:

- `N`, `B`, `T`, `ITERS`. Set at runtime via `argv`.
- `OMEGA`. Default 0.9; set to $\omega_{\text{opt}}(N)$ for `sor2d_rb`.
- `TOL`. Convergence tolerance; default 10^{-5}.
- GPU runtime: `--tile T` and `--halo H`.

List of files

Source files in `src/`:

- `sor2d_cpu.c`, `sor3d_cpu.c`. Single-thread baseline + temporal kernels.
- `sor2d_omp.c`, `sor3d_omp.c`. OMP versions.
- `sor2d_pth.c`, `sor2d_pth_temporal.c`, `sor3d_pth_temporal.c`. Pthreads versions (strip + barrier-per-sweep).
- `sor2d_pth_decomp.c`. Pthreads decomposition study (Figure 7).
- `sor3d_omp_part.c`. 3D OMP partitioning study (slab/pencil/cube).
- `sor2d_rb.c`. Red-black GS-SOR with ω sweep (2.5.2).
- `sor2d_gpu.cu`, `sor3d_gpu.cu`. GPU baseline + shared-memory temporal kernels.